Vector Product and Rotation Simulation

Code Documentation

This document explains the mathematical background and computational steps of the `cross product and vector rotation.py` script shared on GitHub. Using the VPython library, the code calculates the cross product of two vectors (A and B) in three-dimensional space and simulates the rotation of vector A onto vector B.

1. Definition of Vectors and the Cross Product

At the beginning of the code, the x, y, and z components of vectors A and B are obtained from the user. These components are converted into three-dimensional vector objects using VPython's `vector` class.

- **Calculation:** The cross product of the two vectors, $A \times B$, is calculated using the `cross(a, b)` function in the VPython library.
- The resulting vector (named `crossp` in the code) is orthogonal (perpendicular) to the plane formed by vectors A and B, representing the axis of rotation.

2. Finding the Angle Between the Vectors

For vector A to rotate onto vector B, the angle between them must first be known.

- **Calculation:** The angle between the two vectors (in radians) is calculated using the `a.diff_angle(b)` method in the code (assigned to the `ang` variable). Mathematically, this calculation is performed using the dot product.
- In the code, error handling is implemented by checking if the vector magnitudes are zero or if they are parallel/anti-parallel to each other (i.e., when the magnitude of the cross product is very close to zero, $\text{mag}(\text{crossp}) < 10^{-6}$).

3. Finding the Unit Vector of the Rotation Axis

To rotate the vector, we must determine the axis. This axis lies along the direction of the $A \times B$ vector, which is perpendicular to both A and B.

- **Calculation:** The unit vector $u = (u_x, u_y, u_z)$ representing the rotation axis is obtained by dividing the cross product result by its own magnitude (norm).
- In the code, this operation occurs in the following lines:

```
rotaks = crossp / mag(crossp)
ux = rotaks.x
uy = rotaks.y
uz = rotaks.z
```

4. Rotation Matrix Around an Arbitrary Axis in 3D Space

(Rodrigues' Rotation Formula)

The rotation operation, which is the core mathematical component of the code, is executed using a 3D Rotation Matrix based on the Euler-Rodrigues formula. The `Rot(x)` function in the code defines this matrix. The `x` parameter represents the current rotation angle (in radians).

- **Calculation:** The 3×3 matrix that rotates a vector by an angle x around an axis defined by the unit vector $u = (u_x, u_y, u_z)$ is as follows:

$$\begin{bmatrix} u_x^2(1-\cos x)+\cos x & u_x u_y(1-\cos x)-u_z \sin x & u_x u_z(1-\cos x)+u_y \sin x \\ u_x u_y(1-\cos x)+u_z \sin x & u_y^2(1-\cos x)+\cos x & u_y u_z(1-\cos x)-u_x \sin x \\ u_x u_z(1-\cos x)-u_y \sin x & u_y u_z(1-\cos x)+u_x \sin x & u_z^2(1-\cos x)+\cos x \end{bmatrix}$$

- In the code, this matrix is returned as an `np.array` with the help of the NumPy library.

5. Simulation Loop and Matrix Multiplication

During the animation, vector A is rotated step-by-step by small angles to create the simulation.

- **Calculation:** At each time step (`dt`), the rotation matrix is calculated for the current angle `teta`. Vector A (in NumPy array format, `a_np`) is multiplied by this rotation matrix `Rot(teta)`.
- This matrix-vector multiplication is performed in the code using NumPy's `.dot()` method:

```
xrot_np = Rot(teta).dot(a_np)
```

- The resulting new vector (`xrot_np`) is set as the current position of the moving red arrow object (`vecrot.axis`) in the VPython scene. This process is repeated until the `teta` angle reaches the total `ang` value, achieving a smooth rotation animation of the vector within the plane.